

Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИТМО  
ITMO University

Лабораторная работа 4

По дисциплине Объектно-ориентированное программирование

Тема работы Создание и использование методов

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа К3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и  
системы связи

Образовательная программа Программирование в инфокоммуникацион-  
ных системах

Обучающийся

\_\_\_\_\_  
(дата)

\_\_\_\_\_  
(подпись)

Сакулин И.М.  
\_\_\_\_\_  
(Ф.И.О.)

Руководитель

\_\_\_\_\_  
(дата)

\_\_\_\_\_  
(подпись)

Иванов С.Е.  
\_\_\_\_\_  
(Ф.И.О.)

## СОДЕРЖАНИЕ

Стр.

|                               |           |
|-------------------------------|-----------|
| <b>ВВЕДЕНИЕ .....</b>         | <b>3</b>  |
| <b>1 УПРАЖНЕНИЯ 1-3 .....</b> | <b>4</b>  |
| <b>2 УПРАЖНЕНИЕ 4.....</b>    | <b>6</b>  |
| <b>3 УПРАЖНЕНИЕ 5.....</b>    | <b>8</b>  |
| 3.1 Код-ревью.....            | 10        |
| <b>ЗАКЛЮЧЕНИЕ .....</b>       | <b>12</b> |

## ВВЕДЕНИЕ

Отчёт по выполнению лабораторной работы. В тексте работы представлены результаты выполнения пяти упражнений, первые три показывают на примерах как работать с синтаксисом методов, последние два для самостоятельной работы: в них представлена реализация расчёта площади треугольника и решения квадратных уравнений в действительных числах.

**Цель** - изучение и приобретение навыков работы с методами.

## 1 УПРАЖНЕНИЯ 1-3

В проекте был создан класс «Utils», в нём реализованы 3 открытых статических метода «Greater», «Swap», «Factorial» (рисунок 2) и для запуска в «Main» был использован код (рисунок 1).

```
// Упражнение 1
Console.WriteLine("Введите первое число");
int x = int.Parse(Console.ReadLine());
Console.WriteLine("Введите второе число");
int y = int.Parse(Console.ReadLine());
int greater = Utils.Greater(x, y);
Console.WriteLine("Большим из чисел {0} и {1} является {2}", x, y, greater);

// Упражнение 2
Console.WriteLine("Введите первое число");
int x = int.Parse(Console.ReadLine());
Console.WriteLine("Введите второе число");
int y = int.Parse(Console.ReadLine());
Console.WriteLine("До swap: \t{0} {1}", x, y);
Utils.Swap(ref x, ref y);
Console.WriteLine("После swap: \t{0} {1}", x, y);

// Упражнение 3
int f;
Console.WriteLine("Число для факториала");
int x = int.Parse(Console.ReadLine());
bool ok = Utils.Factorial(x, out f);
if (ok)
    Console.WriteLine("Factorial({0}) = {1}", x, f);
else
    Console.WriteLine("Невозможно рассчитать факториал");
```

Рисунок 1 — Тесты класса «Utils.cs» (метод Main)

```

1  using System;
2  namespace Utils
3  {
4      3 references
5      class Utils
6      {
7          1 reference
8          public static int Greater(int a, int b)
9          {
10             if (a > b)
11                 return a;
12             else
13                 return b;
14         }
15
16         1 reference
17         public static void Swap(ref int a, ref int b)
18         {
19             int temp = a;
20             a = b;
21             b = temp;
22         }
23
24         1 reference
25         public static bool Factorial(int n, out int answer)
26         {
27             int f = 1;
28             bool ok = true;
29             try
30             {
31                 checked
32                 {
33                     for (int k = 2; k <= n; ++k)
34                     {
35                         f *= k;
36                     }
37                 }
38             }
39             catch (Exception)
40             {
41                 f = 0;
42                 ok = false;
43             }
44             answer = f;
45             return ok;
46         }
47     }
48 }

```

Рисунок 2 — Класс «Utils.cs»

## 2 УПРАЖНЕНИЕ 4

В проекте был создан класс «Operation», в нём реализованы 2 открытых статических метода «IsRealTriangle» и перегруженный «Area» (рисунок 4) и для запуска в «Main» был использован код (рисунок 3). Программа была протестирована для двух случаев: обычного и равностороннего треугольников (рисунки 5 и 6).

```
using System;
namespace Utils
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            // Упражнение 4
            Console.WriteLine("Треугольник равносторонний? ['+' - да, иначе - нет]: ");
            string ans = Console.ReadLine();
            if (ans == "+")
            {
                Console.WriteLine("Введите длину стороны");
                int a = int.Parse(Console.ReadLine());

                double s = Operation.Area(a);
                Console.WriteLine("Площадь равностороннего треугольника равна {0:f4}", s);
            }
            else
            {
                Console.WriteLine("Введите длину стороны A");
                int a = int.Parse(Console.ReadLine());
                Console.WriteLine("Введите длину стороны B");
                int b = int.Parse(Console.ReadLine());
                Console.WriteLine("Введите длину стороны C");
                int c = int.Parse(Console.ReadLine());

                double s = Operation.Area(a, b, c);
                Console.WriteLine("Площадь треугольника равна {0:f4}", s);
            }
        }
    }
}
```

Рисунок 3 — Тесты класса «Operation.cs»

```

1      using System;
2
3      namespace Utils
4      {
5          2 references
6          internal class Operation
7          {
8              1 reference
9              private static bool IsRealTriangle(double a, double b, double c)
10             {
11                 return a + b > c && a + c > b && b + c > a && a > 0 && b > 0 && c > 0;
12             }
13             1 reference
14             public static double Area(double a, double b, double c)
15             {
16                 if (!IsRealTriangle(a, b, c)) throw new Exception("Такого треугольника не существует!");
17
18                 double p = (a + b + c) / 2;
19                 return Math.Sqrt(p * (p - a) * (p - b) * (p - c)); ;
20             }
21             1 reference
22             public static double Area(double a)
23             {
24                 if (a <= 0) throw new Exception("Такого треугольника не существует!");
25
26                 return Math.Sqrt(3) / 4 * a * a;
27             }
28         }
29     }

```

Рисунок 4 — Класс «Operation.cs»

```

Microsoft Visual Studio Debug Console
Введите длину стороны A
3
Введите длину стороны B
4
Введите длину стороны C
5
Площадь треугольника равна 6,0000

D:\ITMO.PRIVATE\3_OOP\Labs\Lab04\Utils\bin\Debug\Utils.exe (process 31028)
exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 5 — Обычный треугольник

```

Microsoft Visual Studio Debug Console
Треугольник равносторонний? ['+' - да, иначе - нет]: +
Введите длину стороны
4
Площадь равностороннего треугольника равна 6,9282

D:\ITMO.PRIVATE\3_OOP\Labs\Lab04\Utils\bin\Debug\Utils.exe (process 23688)
exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 6 — Равносторонний треугольник

### 3 УПРАЖНЕНИЕ 5

В проекте был создан класс «Solver», в нём реализованы открытый статический метод «QuadraticEquation» (рисунок 8) и для запуска в «Main» был использован код (рисунок 7). Программа была протестирована (рисунок 9).

```
namespace Utils
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            // Упражнение 5
            bool ok;
            Console.WriteLine("Решатель  $a \cdot x^2 + b \cdot x + c = 0$ ");
            Console.WriteLine("Введите a");
            ok = double.TryParse(Console.ReadLine(), out double a);
            if (!ok)
            {
                Console.WriteLine("Разрешены только double через десятичную запятую!");
                return;
            }
            Console.WriteLine("Введите b");
            ok = double.TryParse(Console.ReadLine(), out double b);
            if (!ok)
            {
                Console.WriteLine("Разрешены только double через десятичную запятую!");
                return;
            }
            Console.WriteLine("Введите c");
            ok = double.TryParse(Console.ReadLine(), out double c);
            if (!ok)
            {
                Console.WriteLine("Разрешены только double через десятичную запятую!");
                return;
            }

            int res = Solver.QuadraticEquation(a, b, c, out double x1, out double x2);
            switch (res)
            {
                case -1:
                    Console.WriteLine("Корней уравнения с коэффициентами a = {0}, b = {1}, c = {2} нет.",
                        a, b, c);
                    break;
                case 0:
                    Console.WriteLine("Корень уравнения с коэффициентами a = {0}, b = {1}, c = {2} один x1=x2={3}.",
                        a, b, c, x1);
                    break;
                case 1:
                    Console.WriteLine("Корни уравнения с коэффициентами a = {0}, b = {1}, c = {2} равны x1={3}, x2={4}.",
                        a, b, c, x1, x2);
                    break;
            }
        }
    }
}
```

Рисунок 7 — Тесты класса «Solver.cs»



```

1      using System;
2
3      namespace Utils
4      {
5          0 references
6          internal class Solver
7          {
8              0 references
9              public static int QuadraticEquation(double a, double b, double c, out double x1, out double x2)
10             {
11                 if (a == 0 && b == 0)
12                 {
13                     x1 = 0;
14                     x2 = 0;
15                     return -1;
16                 }
17                 if (a == 0)
18                 {
19                     x1 = -c / b;
20                     x2 = -c / b;
21                     return 0;
22                 }
23                 double D = b * b - 4 * a * c;
24                 if (D < 0)
25                 {
26                     x1 = 0;
27                     x2 = 0;
28                     return -1;
29                 }
30                 else if (D == 0)
31                 {
32                     x1 = -b / (2 * a);
33                     x2 = -b / (2 * a);
34                     return 0;
35                 }
36                 else
37                 {
38                     x1 = (-b + Math.Sqrt(D)) / (2 * a);
39                     x2 = (-b - Math.Sqrt(D)) / (2 * a);
40                     return 1;
41                 }
42             }
43         }
44     }

```

Рисунок 8 — Класса «Solver.cs»

```

Microsoft Visual Studio Debug Console
Решатель a*x^2 + b*x + c = 0
Введите a
1
Введите b
5
Введите c
6
Корни уравнения с коэффициентами a = 1, b = 5, c = 6 равны x1=-2, x2=-3.
D:\ITMO.PRIVATE\3_OOP\Labs\Lab04\Utils\bin\Debug\Utils.exe (process 38688)
exited with code 0 (0x0).
Press any key to close this window . . .

```

Рисунок 9 — Тесты программы

### 3.1 Код-ревью

#### Резюме

Данный код реализует решение квадратного уравнения вида  $(ax^2 + bx + c = 0)$ . Он корректно обрабатывает различные случаи, включая отсутствие корней, один корень и два корня. Однако, есть несколько аспектов, которые можно улучшить для повышения качества кода.

#### Ошибка

Код корректно обрабатывает случаи, когда коэффициенты  $(a)$  и  $(b)$  равны нулю, однако не учитывает ситуацию, когда  $(b)$  равен нулю, а  $(a)$  не равен нулю. В этом случае, уравнение становится линейным, и следует вернуть один корень.

#### Стиль кода

Стиль кода в целом соответствует стандартам C#. Однако, можно улучшить читаемость, добавив комментарии к ключевым частям кода, чтобы объяснить логику решения.

#### Структура кода

Структура кода организована логично, с разделением на классы и методы. Однако, метод QuadraticEquation можно разбить на несколько более мелких методов для улучшения модульности и тестируемости.

#### Читаемость

Читаемость кода хорошая, но можно улучшить форматирование, добавив пробелы между логическими блоками кода. Также стоит использовать более описательные имена переменных, чтобы улучшить понимание кода.

#### Производительность

Производительность кода на текущем уровне приемлема для решения квадратного уравнения. Однако, в случае больших объемов данных или частых вызовов метода, стоит рассмотреть возможность оптимизации.

#### Масштабируемость

Код легко масштабируем, так как его можно расширить для обработки других типов уравнений или добавления дополнительных функций. Однако, для этого потребуются рефакторинг.

#### Безопасность

Код не содержит явных уязвимостей, но стоит учитывать возможность обработки некорректных входных данных. Например, стоит добавить обработку исключений для случаев, когда вводимые данные не могут быть преобразованы в тип `double`.

### **Обработка ошибок**

Обработка ошибок реализована через проверку входных данных и возврат различных кодов состояния. Однако, стоит рассмотреть возможность использования исключений для более гибкой обработки ошибок.

### **Заключение**

В целом, код выполняет свою задачу, но требует некоторых улучшений в области обработки ошибок, читаемости и структуры. Рекомендуется провести рефакторинг для повышения модульности и тестируемости, а также улучшить обработку исключений для повышения надежности.

## **ЗАКЛЮЧЕНИЕ**

В ходе работы было изучено создание и использование методов, были рассмотрены передача и получение данных по ссылкам («ref», «out»), перегрузки. Решены 2 задачи.